알고리즘1 (2026–1)

7. 그리디 알고리즘 (1)

국립금오공과대학교 컴퓨터공학부

컴퓨터공학전공

김 경 수

목차

- 그리디 알고리즘의 개요
- 동전 거스름돈 문제
- 배낭 문제
- 최소 비용 신장 트리

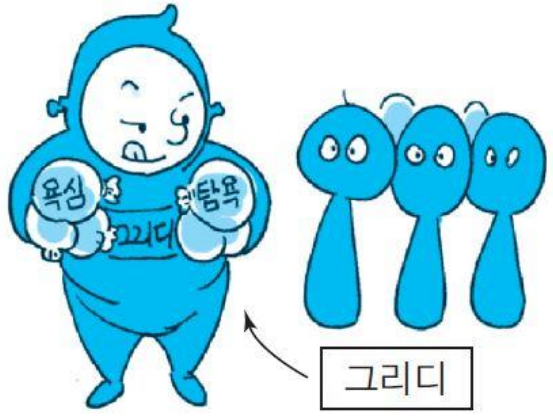
학습목표

- 그리디 알고리즘 패러다임의 기본 개념을 이해하고, 이를 설명할 수 있다.
- 그리디 알고리즘의 특징이 무엇인지 분석하고 그리디 알고리즘의 장/단점을 명확하게 이해할 수 있다.
- 동전 거스름돈 문제의 목표와 제한 조건을 이해하고, 그리디 알고리즘을 이용하여 해를 구할 수 있다.
- 신장 트리와 최소 비용 신장 트리의 정의와 개념을 이해하고, 일반 트리 및 그래프와의 차이점을 명확하게 설명할 수 있다.
- 최소 비용 신장 트리를 생성하는 Kruskal 알고리즘과 Prime 알고리즘의 동작 원리를 그리디 알고리즘의 관점에서 이해하고 실제 그래프가 주어졌을 때, 각 알고리즘을 적용하여 최소 비용 신장 트리를 구할 수 있다.

그리디 알고리즘의 개요

그리디(Greedy) 알고리즘이란?

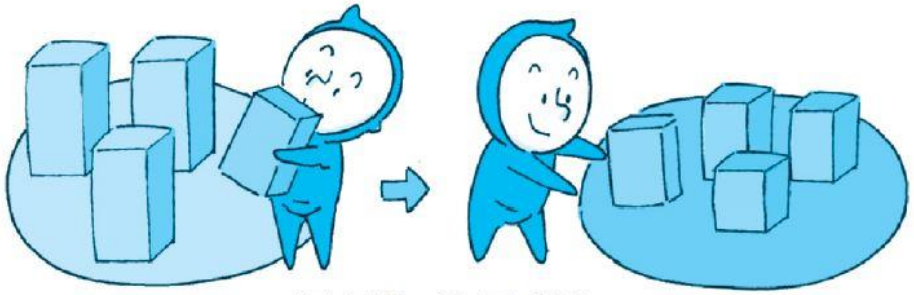
- 그리디 알고리즘은 최적화 문제를 해결하는 알고리즘 패러다임 중 하나이다.
  - ➢최적화(optimization) 문제: 가능한 해들 중에서 가장 좋은 (최대 또는 최소) 해를 찾는 문제
- 탐욕적 방법, 탐욕 알고리즘 등으로도 불린다.
- 그리디 알고리즘에서는 (입력)데이터 간의 관계와 미래의 상황을 고려하지 않고, **현재를 기준으로 가장 좋은 해(최소값 또는 최대값)**를 갖는 데이터 또는 방법을 우선적으로 '욕심내어' 선택한다.
  - ➢이러한 선택을 '**근시안적 선택**'이라고 한다.



그리디(Greedy) 알고리즘

- 그리디 알고리즘은 근시안적인 선택으로 부분적인 최적해를 찾고, 이들을 모아서(축적하여) 문제의 최적해를 얻는다.

최소값을 찾는 문제



가장 작은 데이터 선택

부분해

최대값을 찾는 문제

가장 큰 데이터 선택

그리디(Greedy) 알고리즘

- 그리디 알고리즘은 일단 한 번 선택하면, 이를 절대로 **번복하지 않는다**.
  - 즉, 선택한 데이터를 버리고 다른 것을 취하지 않는다.
- 이러한 특성 때문에 대부분의 그리디 알고리즘들은 매우 단순하며, 주로 제한적인 문제만이 그리디 알고리즘으로 해결된다.
- 그리디 알고리즘으로 항상 최적의 답을 찾을 수 있을까?
  - **문제에 따라 다르다.**

동전 거스름돈 문제

동전 거스름돈 문제

- 동전 거스름돈(Coin Change) 문제를 해결하는 가장 간단하고 효율적인 방법은 무엇인가?
  - ➢ 남은 액수를 초과하지 않는 조건하에 '욕심내어', **즉, 현재 시점에서 가장 큰 액면가**를 갖는 동전을 취하는 것이다.

- 동전 거스름돈 문제의 최소 동전 수를 찾는 그리디 알고리즘
  - ➢ 동전의 종류는 500원, 100원, 50원, 10원, 1원.

동전 거스름돈 문제의 알고리즘

Algorithm CoinChange(W)

– 입력: 거스름돈 금액 W

– 출력: 금액 W를 거슬러 주기 위한 동전의 최소 개수

1. change = W, n500 = n100 = n50 = n10 = n1 = 0; // n500, n100, n50, n10, n1은 받은 동전 개수
2. **while** (change ≥ 500) change = change – 500; n500 = n500 + 1; // 500원 동전의 수 1개 증가
3. **while** (change ≥ 100) change = change – 100; n100 = n100 + 1; // 100원 동전 수 1개 증가
4. **while** (change ≥ 50) change = change – 50; n50 = n50 + 1; // 50원 동전 수 1개 증가
5. **while** (change ≥ 10) change = change – 10; n10 = n10 + 1; // 10원 동전 수 1개 증가
6. **while** (change ≥ 1) change = change – 1; n1 = n1 + 1; // 1원 동전 수 1개 증가
7. **return** n500 + n100 + n50 + n10 + n1; // 총 동전의 수 리턴

CoinChange 알고리즘

- 그리디 알고리즘의 **근시안적 특성**

① CoinChange 알고리즘은 남아있는 거스름돈인 변수 “change”에 대해 가장 높은 가격을 갖는 동전을 거스름돈으로 제공한다.

② 500원 동전을 처리하는 line 2에서는 100원, 50원, 10원, 1원 동전을 몇 개씩 거슬러 주어야 할 것인지에 대해서는 전혀 고려하지 않는다.

③ 500원 동전을 최대한으로 거슬러 준 후, line 3에서는 현재 남아있는 거스름돈 “change”를 거슬러주기 위해 100원 동전을 최대 몇 개까지 사용할 수 있는지 계산하며, 이 때에도 나머지 50원, 10원, 1원 동전에 대해서는 전혀 고려하지 않는다.

④ 이후 50원, 10원, 1원 동전에 대해서도 같은 방법으로 동전 개수를 계산한다.

CoinChange 알고리즘의 수행 과정

760원의 거스름돈 반환을 위한 그리디 알고리즘 예제

CoinChange 알고리즘의 문제점

- 만약 한국은행에서 160원 동전을 <u>추가로</u> 발행한다면, CoinChange 알고리즘이 항상 동전의 최소 개수를 계산할 수 있을까?
  - ➢ 거스름돈이 200원이라면, CoinChange 알고리즘은 **160원 동전 1개**와 **10원 동전 4개**로서 **총 5개를 리턴**

CoinChange 알고리즘의 결과

CoinChange 알고리즘의 문제점

➢ 그러나 200원에 대한 최소 동전 수는 100원짜리 동전 2개이다.

CoinChange 알고리즘의 결과

최소 동전의 거스름돈

- 결과적으로, CoinChange 알고리즘의 결과가 항상 최적의 해결책(답)임을 보장하지는 **못함**.
  - ➢ 한편, 한국은행에서는 실생활에서 거스름돈을 계산하기 위해 그리디 알고리즘을 적용할 수 있도록 동전의 종류를 지정하여 발행하고 있음.

배낭 문제

배낭 문제의 정의와 종류

- **배낭(Knapsack) 문제**
  - ➢ n개의 물건이 각각 1개씩 있고,
  - ➢ 각 물건은 무게와 가치를 가지고 있으며,
  - ➢ 배낭이 한정된 무게의 물건들을 담을 수 있을 때,
  - ➢ 최대의 가치를 갖도록 배낭에 넣을 물건들을 정하는 문제이다.

배낭 문제의 정의와 종류

- **부분 배낭(Fractional Knapsack) 문제**
  - 물건을 **부분적으로 담는 것**을 허용한다.
  - 그리디 알고리즘으로 해결 가능하다.
- **0-1 배낭(0-1 Knapsack) 문제**
  - 부분 배낭 문제의 원형으로 물건을 **통째로** 배낭에 넣어야 한다.
  - 동적 계획 알고리즘, 백트래킹 기법, 분기 한정 기법으로 해결할 수 있다.

부분 배낭 문제의 아이디어

- 부분 배낭 문제에서는 물건을 부분적으로 배낭에 담을 수 있으므로, 최적해를 위해서 '욕심을 내어' **단위 무게 당 가장 값나가는 물건**을 배낭에 넣고, 계속해서 **그 다음으로 값나가는 물건**을 넣는다.
- 만일 물건을 '통째로' 배낭에 넣을 수 없으면, 배낭에 넣을 수 있을 만큼만 **물건을 부분적으로 나눠서** 배낭에 담는다.

부분 배낭 문제 알고리즘

Algorithm FractionalKnapsack()

– 입력: n개의 물건, 각 물건의 무게와 가치, 배낭의 용량 C

– 출력: 배낭에 담은 물건 리스트 L과 배낭 속의 물건 가치의 합 v

1. 각 물건에 대해 단위 무게 당 가치를 계산한다;
2. S = 물건들을 단위 무게 당 가치를 기준으로 내림차순으로 정렬한다;
3. L = ∅; w = 0; v = 0;

   // L은 배낭에 담을 물건 리스트, w는 배낭에 담긴 물건들의 무게의 합, v는 배낭에 담긴 물건들의 가치의 합

4. S에서 단위 무게 당 가치가 가장 큰 물건 x를 가져온다;

부분 배낭 문제 알고리즘

```
while (w + (x의 무게) ≤ C) do
    x를 L에 추가한다;
    w = w + (x의 무게);
    v = v + (x의 가치);
    x를 S에서 제거한다;
    S에서 단위 무게 당 가치가 가장 큰 물건 x를 가져온다;
if ( C – w > 0 ) then   // 배낭에 물건을 부분적으로 담을 여유가 있으면
    물건 x를 (C – w) 만큼만 L에 추가한다;
    v = v + (C – w) 만큼의 x의 가치;
return L, v;
```

부분 배낭 문제 알고리즘의 수행 과정

- 배낭의 최대 용량 = 40그램
- 단위 무게 당 가치로 정렬: S = [백금, 금, 은, 주석]

물건	단위 그램당 가치
백금	6만원
금	5만원
은	4천원
주석	1천원

부분 배낭 문제 알고리즘의 수행 과정

- 백금을 통째로 담는다.
  - ➢ 배낭에 담긴 물건(들)의 무게 w = 10, 얻는 가치 v = 60

- 금을 통째로 담는다.
  - ➢ 배낭에 담긴 물건(들)의 무게 w = 25, 얻는 가치 v = 60+ 75

    = 135

부분 배낭 문제 알고리즘의 수행 과정

10만원

- "은"도 전부 담고 싶다. 하지만....
  - ➢ 배낭에 담긴 물건(들)의 무게가 w = 25 + 25 = 50이 되어 배낭 용량 초과한다.
  - ➢ 이에 따라, "은"은 40 – 25 = 15 만큼만 담는다.

- 따라서 배낭에 담긴 물건(들)의

  총 무게는 w = 40 이고,

  얻는 가치는 v = 135 + (0.4 × 15) = 141만원이다.

부분 배낭 문제 알고리즘의 시간 복잡도

- **Line 1:** n개의 물건 각각의 단위 무게 당 가치를 계산하는 데는 $\boldsymbol{O(n)}$ 시간 소요
- **Line 2:** 물건의 단위 무게 당 가치에 대해서 정렬하기 위해 $\boldsymbol{O(n \log n)}$ 시간 소요
- **Line 5~10:** while–루프의 수행은 n번을 넘지 않으며, 루프 내부의 수행은 $\boldsymbol{O(1)}$ 시간 소요
- **Line 11~14:** 각각 $\boldsymbol{O(1)}$ 시간 소요
- **알고리즘의 총 시간 복잡도:**

$$O(n) + O(n \log n) + \big(n \times O(1)\big) + O(1) = \boldsymbol{O(n \log n)}$$

배낭 문제 알고리즘의 응용

- 0–1 배낭 문제는 최소의 비용으로 자원을 할당하는 문제로서, 조합론, 계산이론, 암호학, 응용 수학 분야에서 기본적인 문제로 다뤄진다.
  - ✓ "버리는 부분을 최소화"하기 위한 원자재 자르기 문제
  - ✓ 자산투자 및 금융 포트폴리오에서의 최선의 선택 문제
  - ✓ Merkle–Hellman 배낭 암호 시스템에 사용

$a = [a_1, a_2, \ldots, a_k]$ and $x = [x_1, x_2, \ldots, x_k]$.

$$\boxed{s = knapsackSum\,(a, x) = x_1 a_1 + x_2 a_2 + \cdots + x_k a_k}$$

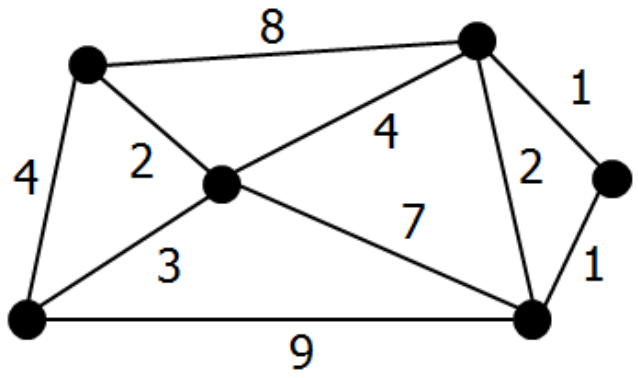
주어진 a와 x로부터 s를 계산하는 것은 매우 쉽다. 하지만 s와 a가 주어졌다고 했을 때, x를 구하는 것은 어렵다.

◀ 배낭 암호의 기본 원리

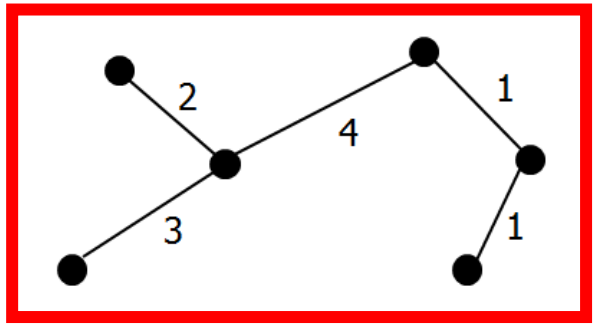
최소 비용 신장 트리

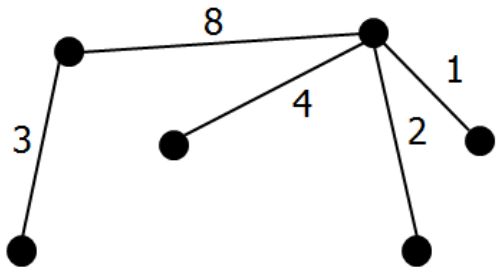
최소 비용 신장 트리

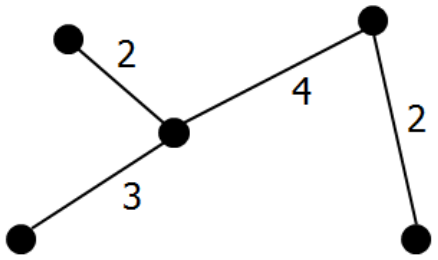
- **최소 비용 신장 트리 (Minimum Cost Spanning Tree: MST)**
  - ➢ 주어진 가중치 그래프에서 **사이클이 없이** **모든 점들을 연결**시킨 트리 중에서 간선들의 **가중치 합이 최소**인 트리



  - ➢ Q. 다음 중 위 그래프의 최소 비용 신장 트리는?

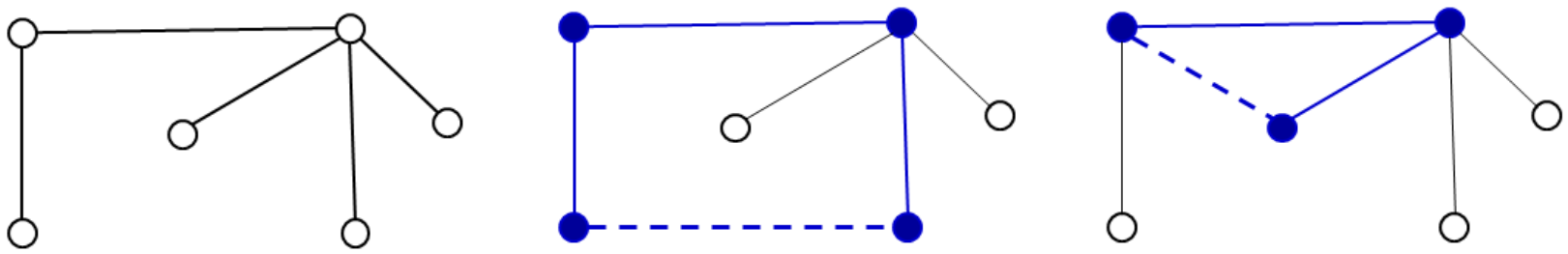






최소 비용 신장 트리

- 최소 비용 신장 트리의 특징
  - ➢ 사이클이 발생하지 않도록 모든 정점을 연결한다.
  - ➢ 위의 조건을 만족하면서 동시에 간선 비용의 합이 최소가 되어야 한다.
- 그래프 내의 정점의 수 = n
  - ➢ 신장 트리에는 정확히 n – 1개의 간선이 있다.
  - ➢ 트리에는 간선을 하나 추가시키면, 반드시 사이클이 생성된다.



트리 점선으로 된 선분을 추가하여 만들어진 사이클

Kruskal의 MST 알고리즘

- 최소 비용 신장 트리가 최소 비용의 간선으로 구성됨과 동시에 사이클을 포함하지 않는다는 조건에 근거하여 고안된 알고리즘
- 각 단계에서 **사이클을 이루지 않는** **최소 가중치를 갖는 간선**을 선택한다.
- Kruskal 알고리즘의 MST 구축 순서

① 그래프 내의 모든 간선들을 가중치에 따라 오름차순으로 정렬한다.

② 정렬된 간선들의 리스트에서 맨 앞에 위치한 간선을 선택한다.

③ 선택한 간선이 사이클을 생성하는 지 확인한 후 사이클을 생성하지 않으면 해당 간선을 그래프에 추가한다.

④ 선택한 간선이 사이클을 생성하는 경우 그 간선은 제외하고 ②의 과정을 반복한다.

Kruskal의 MST 알고리즘

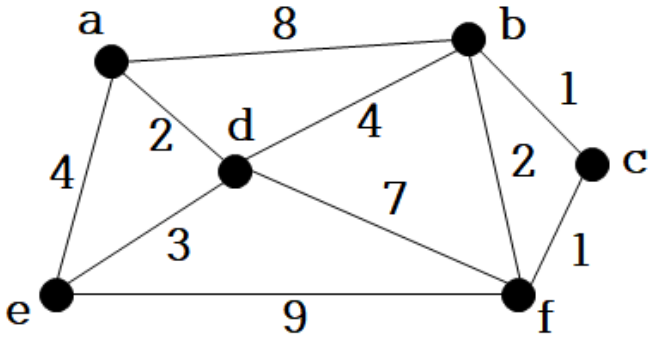
Algorithm KruskalMST(G)

– 입력: 가중치 그래프 G=(V, E) // 정점의 개수 n = | V |, 간선의 개수 m = | E |

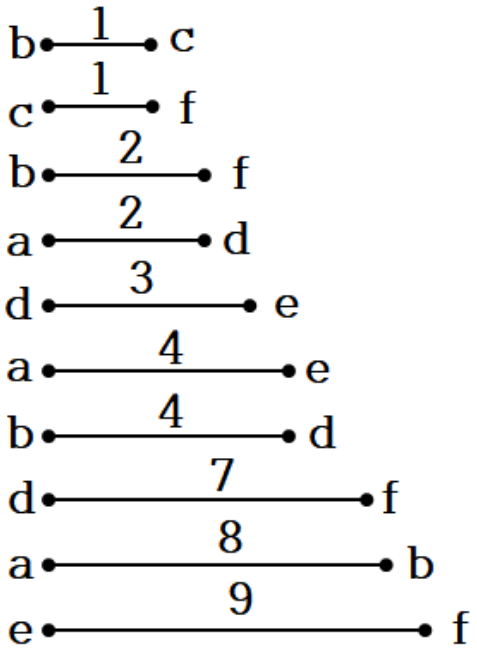
– 출력: 최소 비용 신장 트리 T

```
L = 가중치의 오름차순으로 간선들을 정렬한 간선 리스트;
T = ∅; // 트리 T를 empty set으로 초기화
while ( T의 간선 수 < n –1 ) do
    L에서 가장 작은 가중치를 가진 간선 e를 가져오고, e를 L에서 제거한다;
    if ( 간선 e가 T에 추가되어 사이클을 만들지 않으면 ) then
        e를 T에 추가한다;
    else   // e가 T에 추가되어 사이클이 생기는 경우
        e를 버린다;
return T;   // T는 최소 비용 신장 트리
```

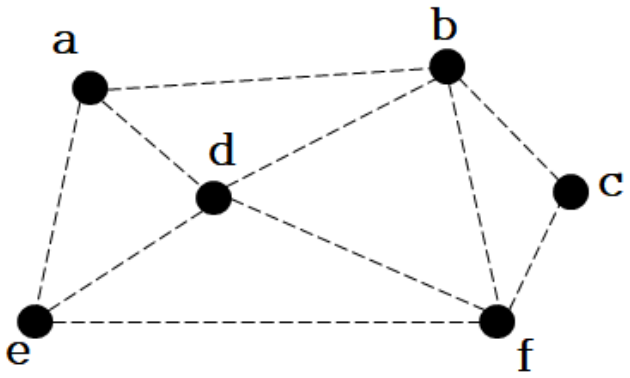
Kruskal 알고리즘의 수행 과정 예시 (1/8)



입력 그래프



정렬된 리스트 L

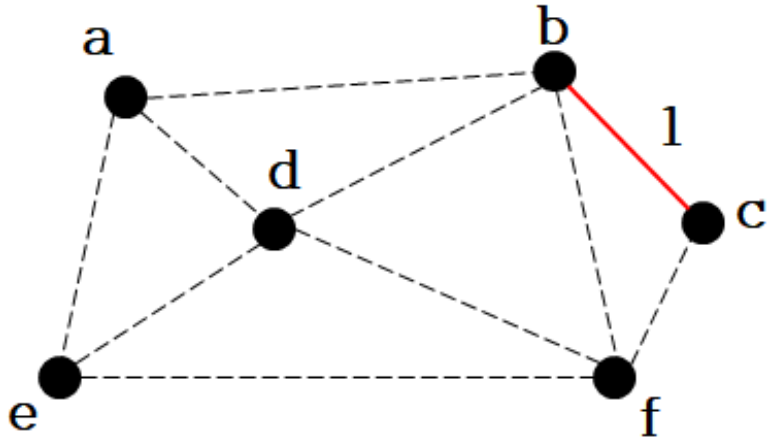


T = ∅

Kruskal 알고리즘의 수행 과정 예시 (2/8)

리스트 L

b	1	c
c	1	f
b	2	f
a	2	d
d	3	e
a	4	e
b	4	d
d	7	f
a	8	b
e	9	f

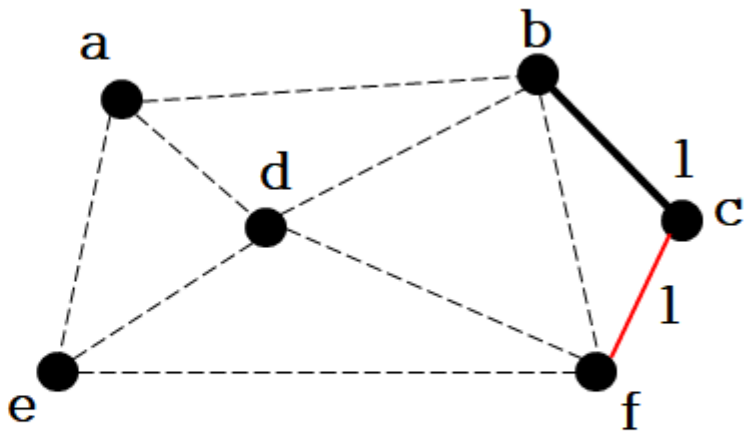


간선 (b, c) 추가

Kruskal 알고리즘의 수행 과정 예시 (3/8)

리스트 L

c	1	f
b	2	f
a	2	d
d	3	e
a	4	e
b	4	d
d	7	f
a	8	b
e	9	f

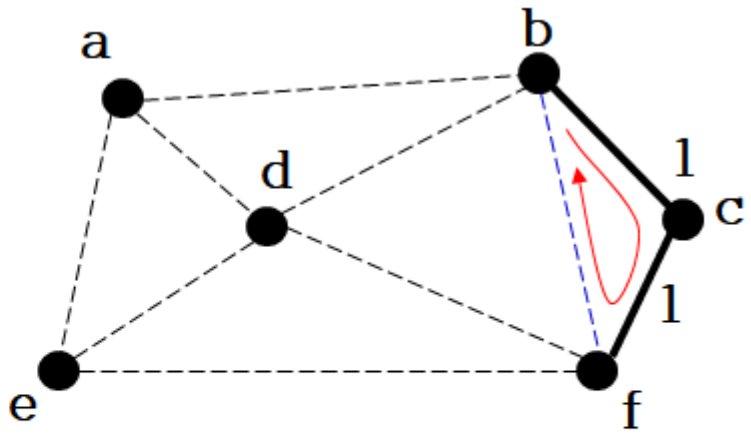


간선 (c, f) 추가

Kruskal 알고리즘의 수행 과정 예시 (4/8)

리스트 L

b–f	2
a–d	2
d–e	3
a–e	4
b–d	4
d–f	7
a–b	8
e–f	9



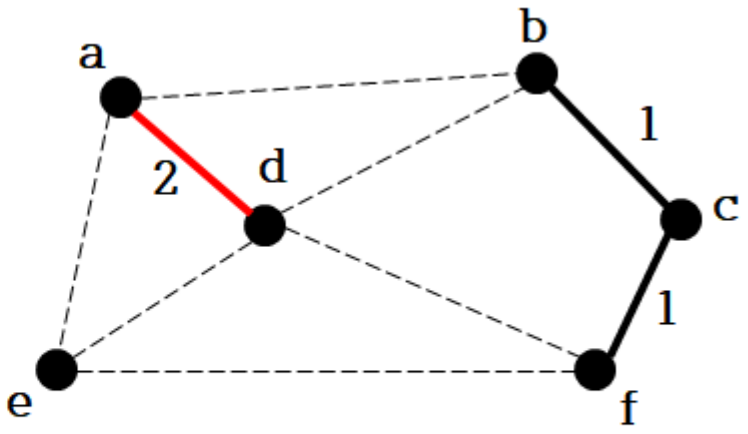
사이클 b–c–f–b

간선 (b, f) 버림

Kruskal 알고리즘의 수행 과정 예시 (5/8)

리스트 L

2	a	d
3	d	e
4	a	e
4	b	d
7	d	f
8	a	b
9	e	f

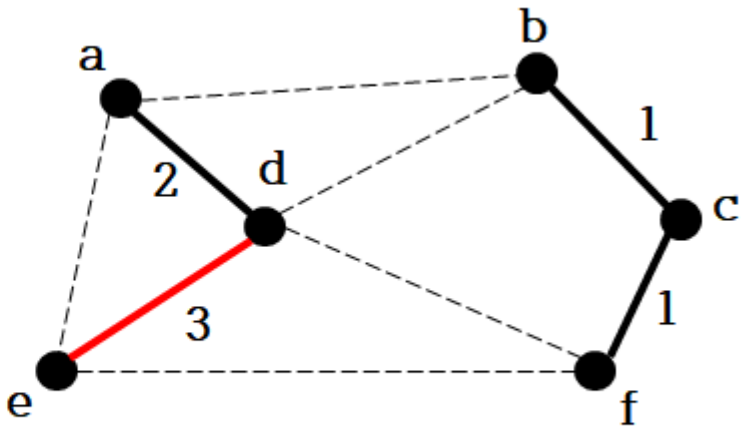


간선 (a, d) 추가

Kruskal 알고리즘의 수행 과정 예시 (6/8)

리스트 L

3	d	e
4	a	e
4	b	d
7	d	f
8	a	b
9	e	f

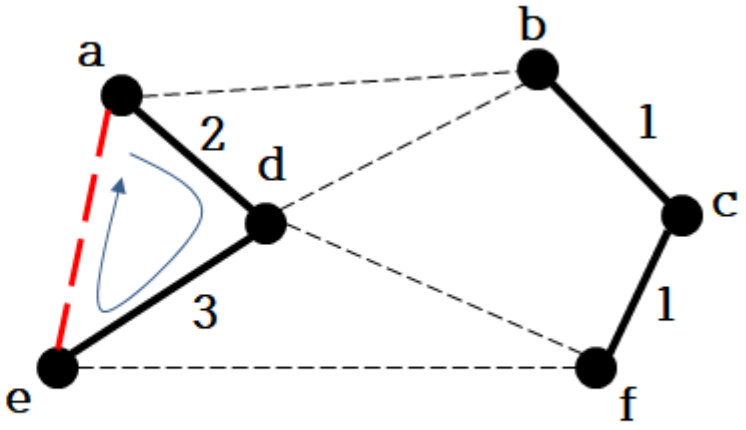


간선 (d, e) 추가

Kruskal 알고리즘의 수행 과정 예시 (7/8)

리스트 L

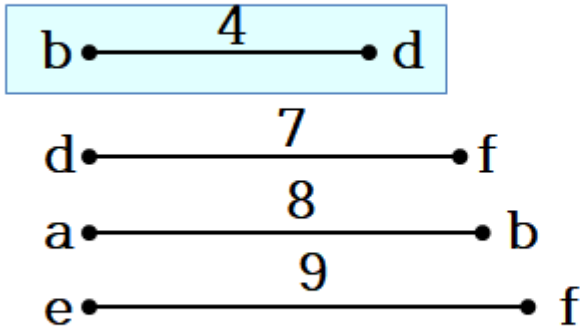
4	a	e
4	b	d
7	d	f
8	a	b
9	e	f

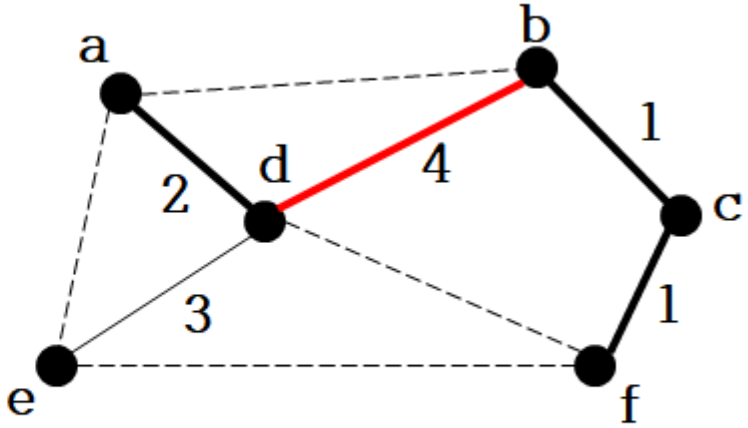


사이클 a-d-e-a

Kruskal 알고리즘의 수행 과정 예시 (8/8)

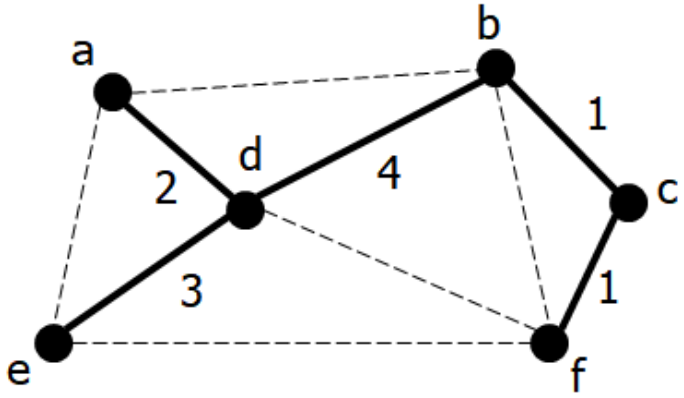
리스트
L





간선 (b, d) 추가

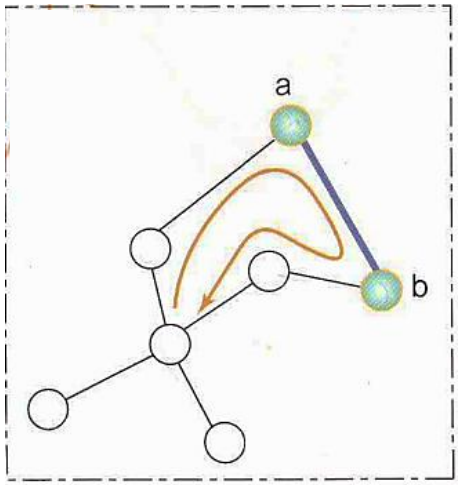
최종 MST



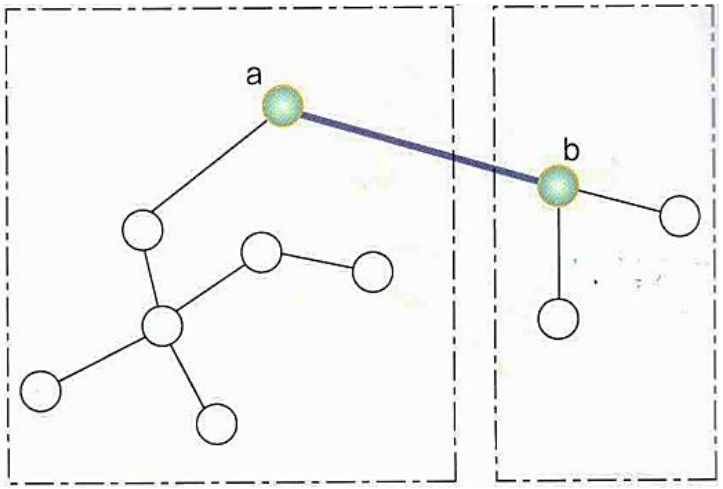
Kruskal 알고리즘에서 생각해 볼 문제

- 새로운 간선을 추가할 때 사이클의 발생 여부 확인 방법은?

(A)



(B)

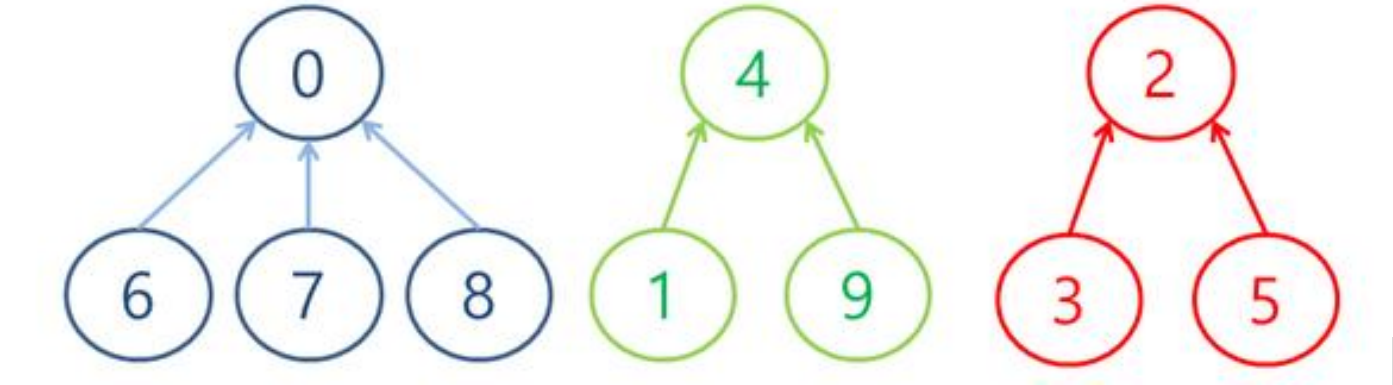


- **(A)** 간선의 양 끝 정점 모두 같은 집합에 속하면 해당 간선을 추가하였 때 사이클이 형성됨.
- **(B)** 간선이 양 끝 정점이 서로 다른 집합에 속하는 경우 해당 간선을 추가해도 사이클이 형성되지 않음.

- 이를 효과적으로 수행하기 위한 방법 → **Union-find 연산**

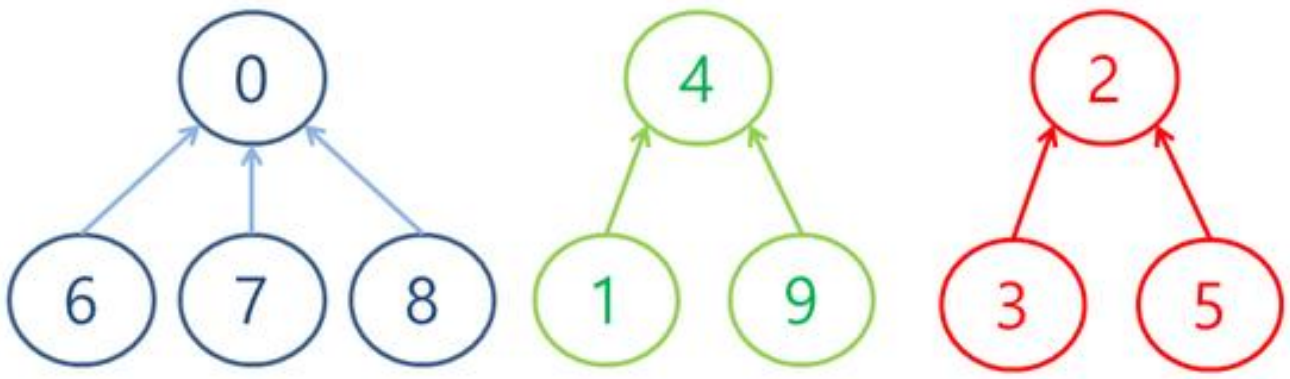
Union-find 연산

- 임의의 분리 집합(disjoint set)을 표현하기 위해 트리를 사용
  - 집합의 표현 → **트리**
  - **Union(set_1, set_2)** → 두 집합 set_1, set_2를 나타내는 트리들을 하나로 연결
  - **Find(e)** → 주어진 원소 "e"가 속하는 집합의 대표 원소를 반환
  - 집합의 구분 → **루트 노드의 번호**로 구분 (※ 루트 노드의 값 = 집합의 대표 원소)
- 집합 내 원소의 탐색 속도를 향상시키기 위해서 모든 노드들을 루트 노드의 바로 아래에 위치시킴.



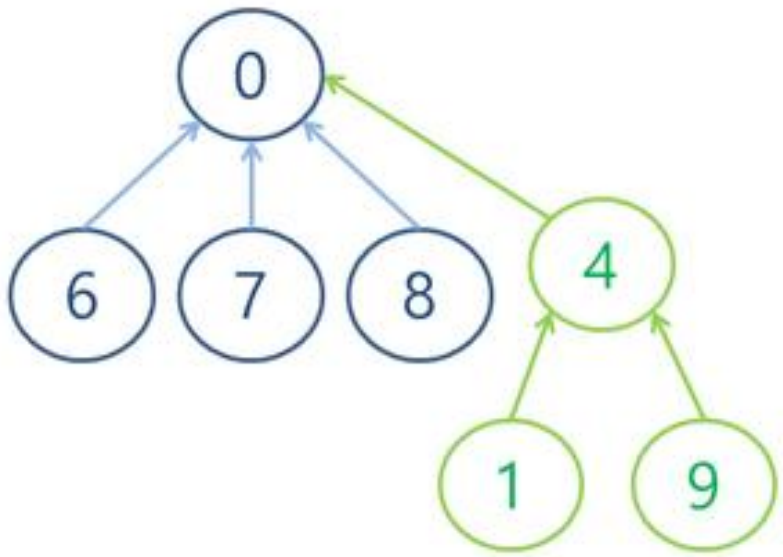
Union-find 연산

집합의 표현 방법



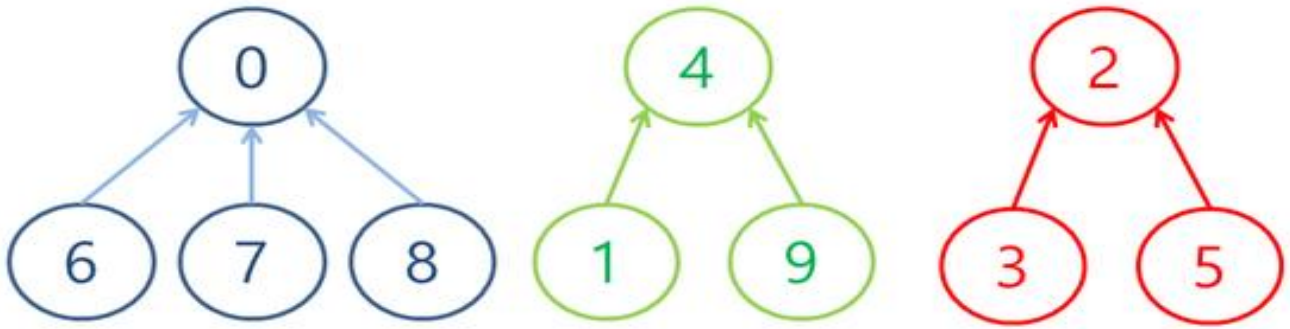
Set_1 Set_2 Set_3

합집합 (union)

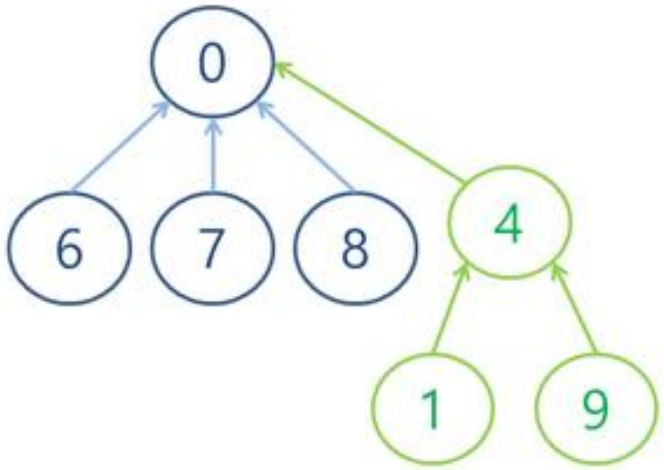


Set_1 ∪ Set_2

Union-find 연산 예제



Set_1 Set_2 Set_3



Set_1 ∪ Set_2

- find(7) = 0
- find(1) = 4
- find(3) = 2
- find(2) = 2
- set_1 = 0
- set_2 = 4
- union(set_1, set_2)

Kruskal 알고리즘과 Union-find 연산

Algorithm KruskalMST(G)

– 입력: 가중치 그래프 G=(V, E) // 정점의 개수 n = | V |, 간선의 개수 m = | E |

– 출력: 최소 비용 신장 트리 T;

1. L = 가중치의 오름차순으로 간선들을 정렬한 간선 리스트;
2. T = Φ; // 트리 T를 empty set으로 초기화
3. **while** (T의 간선 수 < n – 1) **do**
4. L에서 가장 작은 가중치를 가진 간선 e를 가져오고, e를 L에서 제거한다;
5. **if (간선 e가 T에 추가되어 사이클을 만들지 않으면) then**
6. e를 T에 추가한다;
7. **else** // e가 T에 추가되어 사이클이 생기는 경우
8. e를 버린다;
9. **return** T; // T는 최소 비용 신장 트리

임의의 간선 추가 시 사이클이 발생하는지 여부를 검사하는 부분을 "union-find" 연산을 사용하는 것으로 구현할 수 있다.

```
[v1, v2] = 간선 e의 양 끝 정점;
v1_set = find(v1);
v2_set = find(v2);
if ( v1_set != v2_set ) then
    e를 T에 추가한다;
    union(v1_set, v2_set);
else
    e를 버린다;
```

Kruskal 알고리즘의 시간 복잡도

- **Line 1:** m개의 간선 정렬을 위한 시간 복잡도: $\boldsymbol{O(m \log m)}$
- **Line 2:** T를 초기화하는데 소요되는 시간 복잡도: $\boldsymbol{O(1)}$
- **Line 3~8:** while 반복문은 최대 $\boldsymbol{m}$번 수행된다.
  - while–루프 내에서는 L로부터 가져온 간선 e가 사이클을 만드는지 검사하는데 소요되는 시간 복잡도는 대략 $\boldsymbol{O(1)}$ 이다.
  - 즉, while 반복문을 수행하는데 소요되는 최대 시간 복잡도는 $\boldsymbol{O(m)}$이다.
  - 한편, 간선의 개수가 m개라면, 정점의 개수 n은 $m + 1 = n$ 로 표현 가능하다.
  - 즉, 간선의 개수 m에 대한 시간 복잡도를 정점의 개수 n에 대한 시간 복잡도로 표현할 수 있다.
- 따라서 Kruskal 알고리즘의 시간 복잡도는 $\boldsymbol{O(n \log n)}$ 이다.

Prim의 MST 알고리즘

- 시작 정점에서 출발하여 신장 트리 집합을 단계적으로 확장해 나가는 방법
- Kruskal 알고리즘과는 달리 그래프의 **정점을 추가**하는 과정을 통해서 최소 비용 신장 트리를 구축함.
- Prim 알고리즘의 MST 구축 순서
  ① 시작 단계에서는 시작 정점만이 신장 트리 집합에 포함된다.
  ② 앞 단계에서 만들어진 신장 트리 집합에, 인접한 정점들 중에서 최소 가중치 간선으로 연결된 정점을 선택하여 트리를 확장한다.
  ③ 이 과정을 트리가 $n-1$개의 간선을 가질 때까지 계속 반복한다.

Prim의 MST 알고리즘

Algorithm PrimMST(G)

– 입력: 가중치 그래프 G=(V, E); // 정점의 개수 n = | V |, 간선의 개수 m = | E |

– 출력: 최소 비용 신장 트리 T;

1. 그래프 G에서 임의의 정점 p를 시작점으로 선택한 후 D[p] = 0으로 설정한다;

   // D[v]는 T에 속하는 정점 u와 정점 v를 연결하는 간선의 최소 가중치를 저장하기 위한 배열 D의 원소

2. **for** (정점 p가 아닌 각 정점 v에 대하여) **do**

3. **if** (간선 (p, v)가 그래프에 있으면) **then**

4. D[v] = 간선 (p, v)의 가중치;

5. **else**

6. D[v] = ∞;

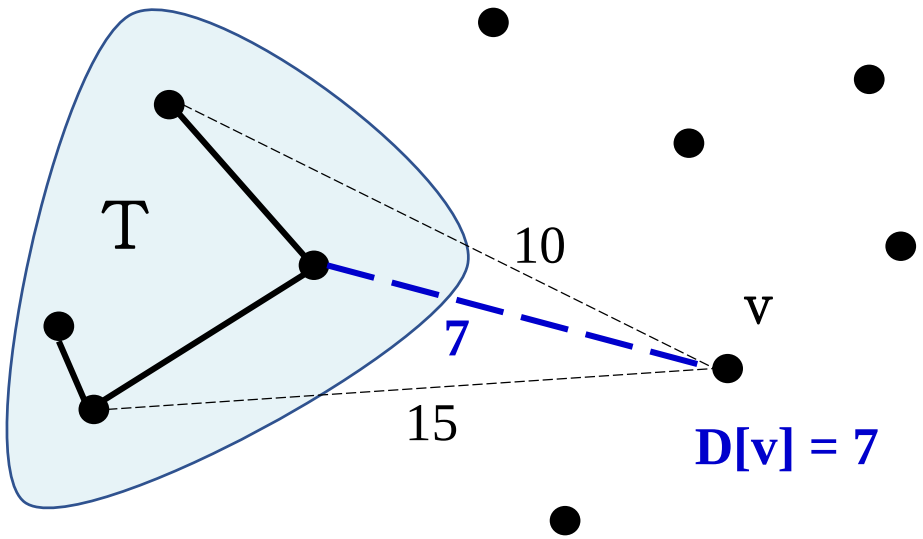
Prim의 MST 알고리즘

```
T= { p };  // 초기에 트리 T는 정점 p만을 갖는다.
while ( T에 속하는 정점의 수 < n ) do
        T에 속하지 않은 각 정점 v에 대하여, D[v]가 최소인 정점 v_min과 연결된 간선 (u, v_min)을
    T에 추가한다; 단, u는 T에 속한 정점이며, 이때 정점 v_min도 T에 추가된다;
      for ( T에 속하지 않은 각 정점 w에 대해서 ) do
          if ( 간선 (v_min, w)의 가중치 < D[w] ) then
                D[w] = 간선 (v_min, w)의 가중치;  // D[w]를 갱신한다.
return T;   // T는 최소 비용 신장 트리이다.
```

Prim 알고리즘의 수행 과정 예시 (1/11)

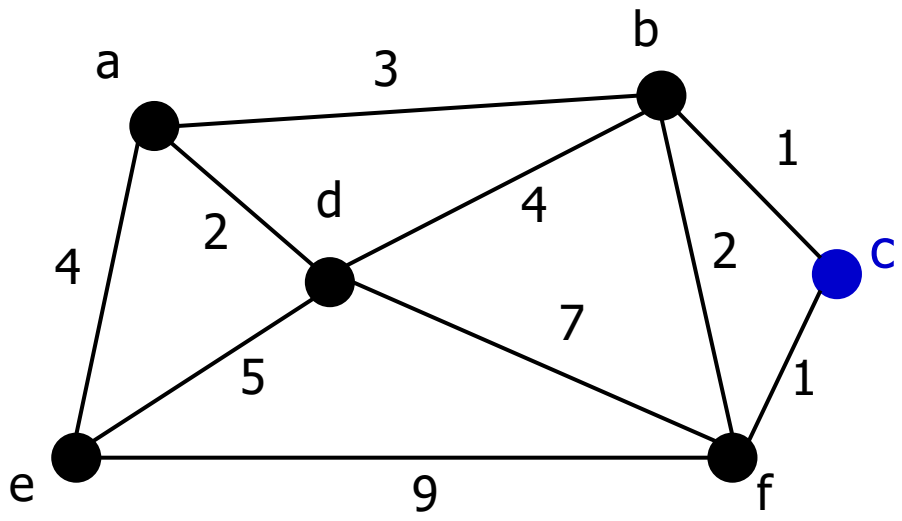
✓ D[v]는 T에 속하는 정점 u와 정점 v를 연결하는 간선의 최소 가중치를 저장하기 위한 배열 D의 원소

- D[v] 계산
  - ➢ 임의로 정점 p를 선택하고, D[p] = 0으로 설정한다.
  - ➢ D[v]에는 정점 v와 T에 속한 정점들을 연결하는 간선들 중에서 최소 가중치를 가진 간선의 가중치를 저장한다.
  - ➢ (예) 아래 그래프에서 D[v]에는 10, 7, 15 중에서 최소 가중치인 **7**이 저장된다.



Prim 알고리즘의 수행 과정 예시 (2/11)

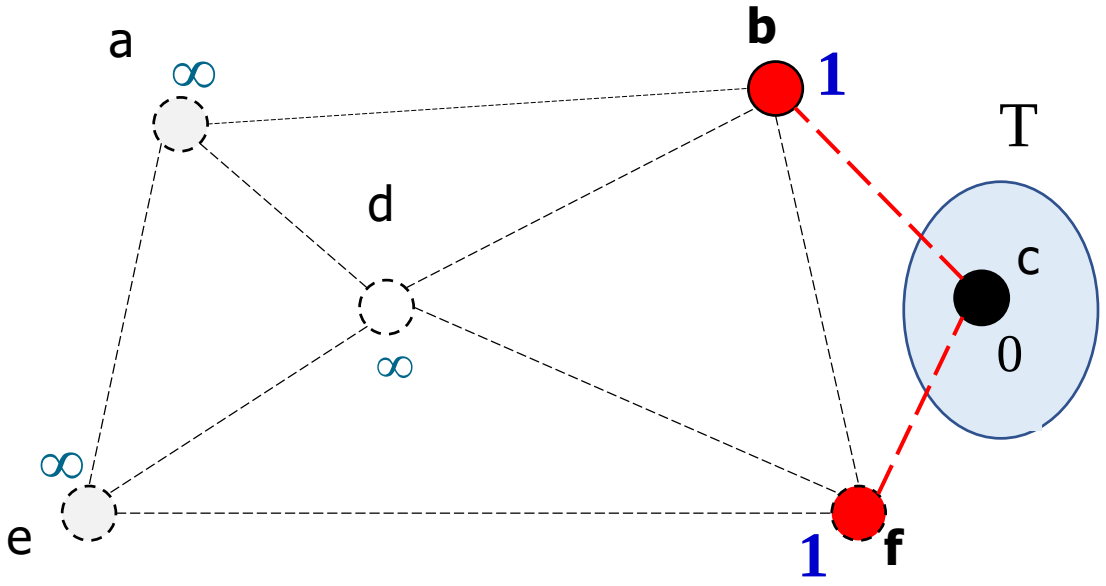
- **Line 1:** 임의의 정점 "c" 를 선택하고 D[c]=0으로 초기화



Vertex	a	b	c	d	e	f
Distance	∞	∞	**0**	∞	∞	∞

Prim 알고리즘의 수행 과정 예시 (3/11)

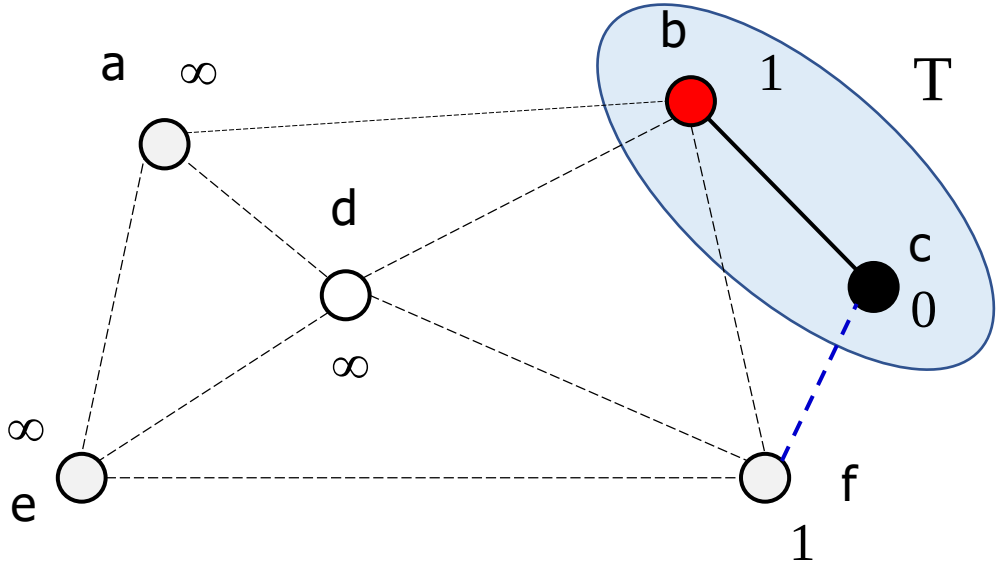
- **Line 2 – 6:**
  - 시작 정점 c와 간선으로 연결된 각 정점 v에 대해서, D[v]를 각 간선의 가중치로 초기화
  - 정점 c와 연결되지 않은 나머지 정점 v에 대해서는 D[v] = ∞로 초기화



Vertex	a	b	c	d	e	f
Distance	∞	1	0	∞	∞	1

Prim 알고리즘의 수행 과정 예시 (4/11)

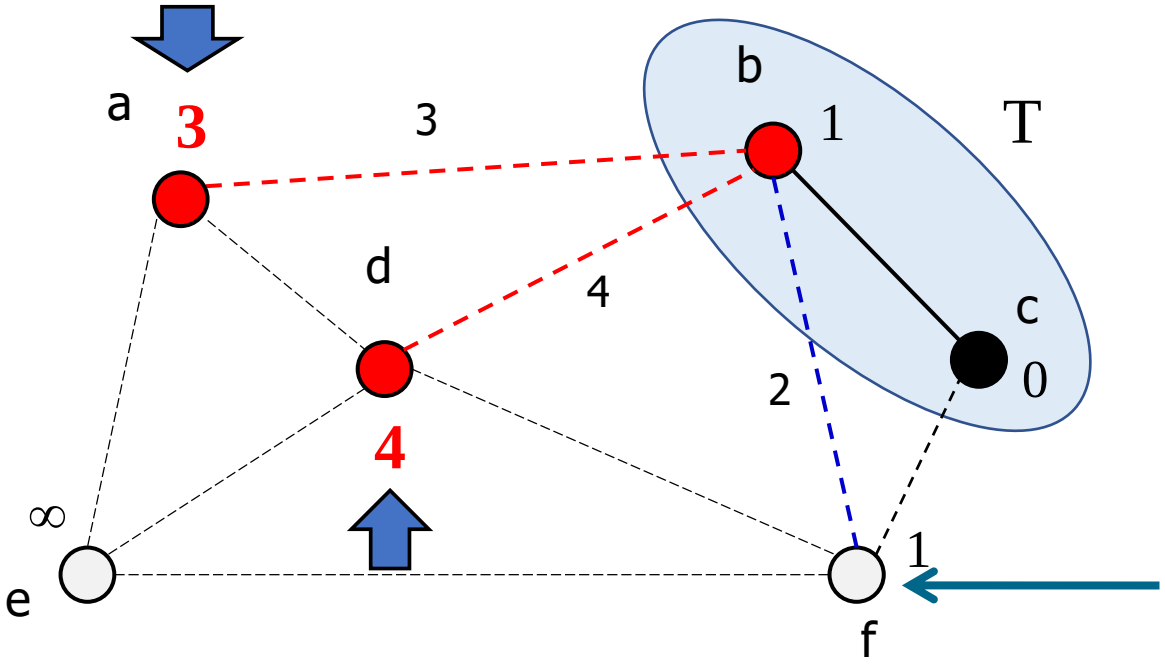
- **Line 7:** T={ c }로 초기화한다.
- **Line 8 – 12:**
  - T에 가장 가까운 정점 b를 추가한다.



Vertex	a	b	c	d	e	f
Distance	∞	1	0	∞	∞	1

Prim 알고리즘의 수행 과정 예시 (5/11)

- 정점 b에 연결된 정점 a와 d에 대한 거리 D[a]와 D[d] 갱신한다.
- 한편, 정점 f에 대해서는 정점 b와 연결되어 있음에도 불구하고, D[f]를 갱신하지 않는다.

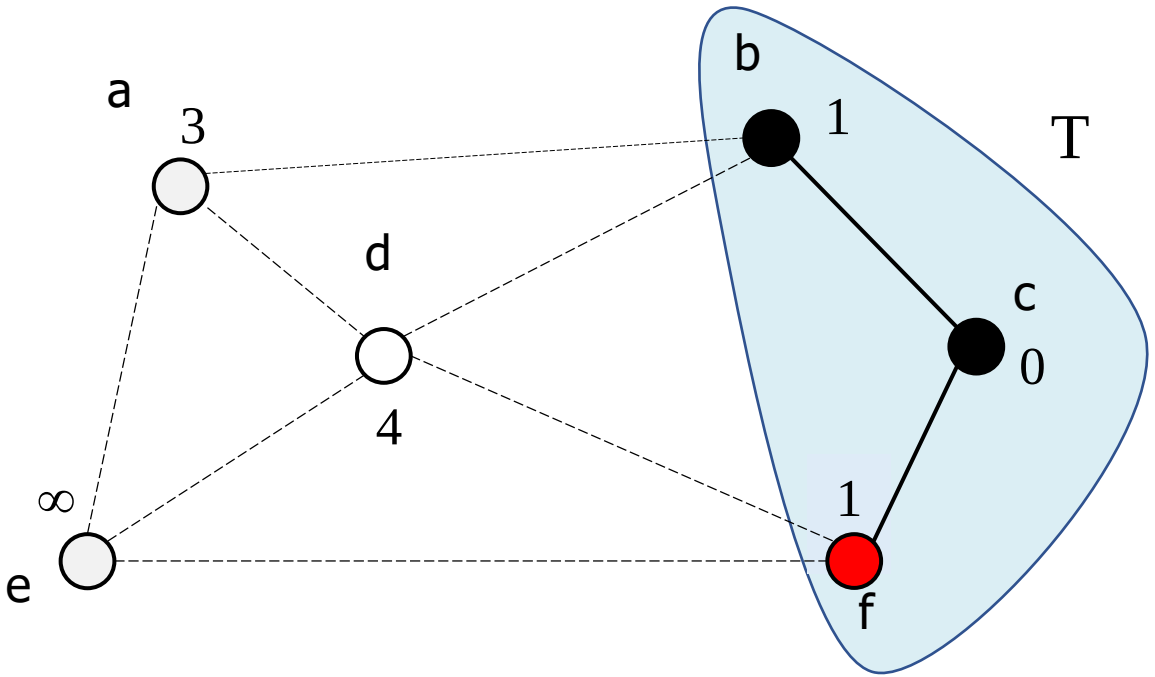


정점 b와 f를 잇는 간선의 가중치(2)가 c와 f를 잇는 가중치(1)보다 크므로 갱신되지 않음.

Vertex	a	b	c	d	e	f
Distance	**3**	1	0	**4**	∞	1

Prim 알고리즘의 수행 과정 예시 (6/11)

- T와 가장 가까운 정점 f를 T에 추가한다.

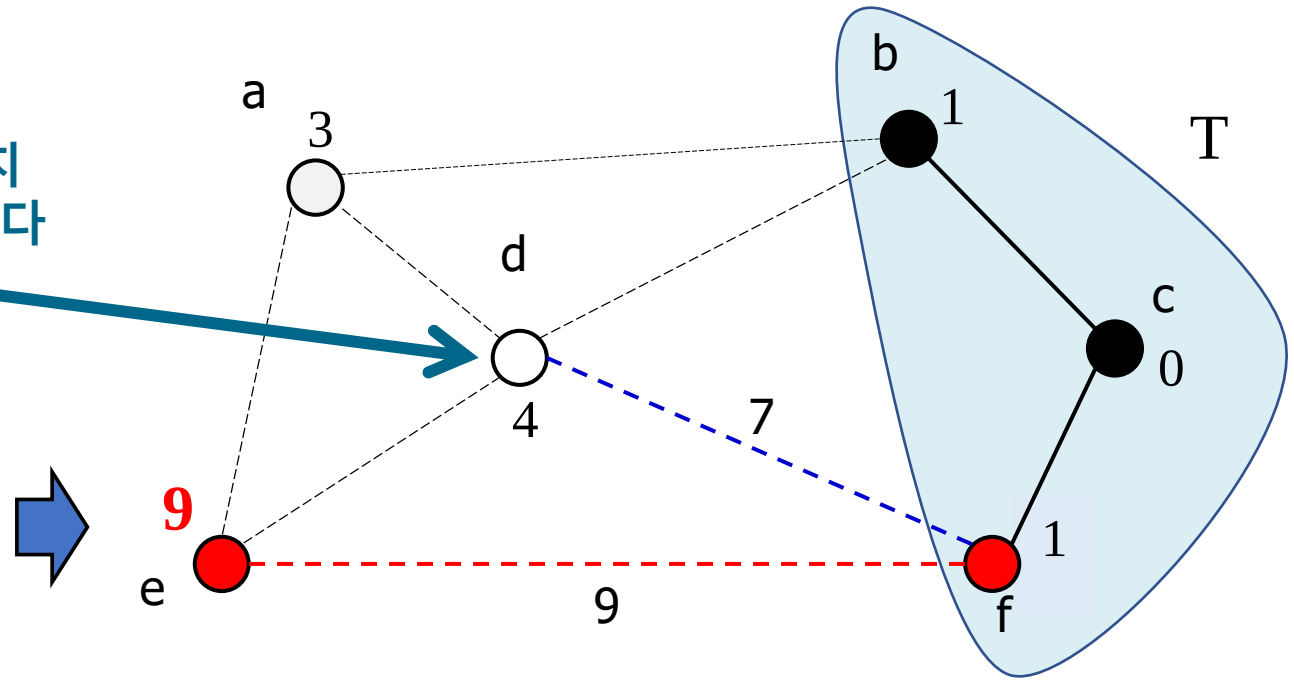


Vertex	a	b	c	d	e	f
Distance	3	1	0	4	∞	1

Prim 알고리즘의 수행 과정 예시 (7/11)

- 정점 f에 연결된 정점 e의 거리 D[e]를 갱신한다.
- 한편, 정점 f에 연결된 정점 d의 거리 D[d]는 갱신되지 않는다.

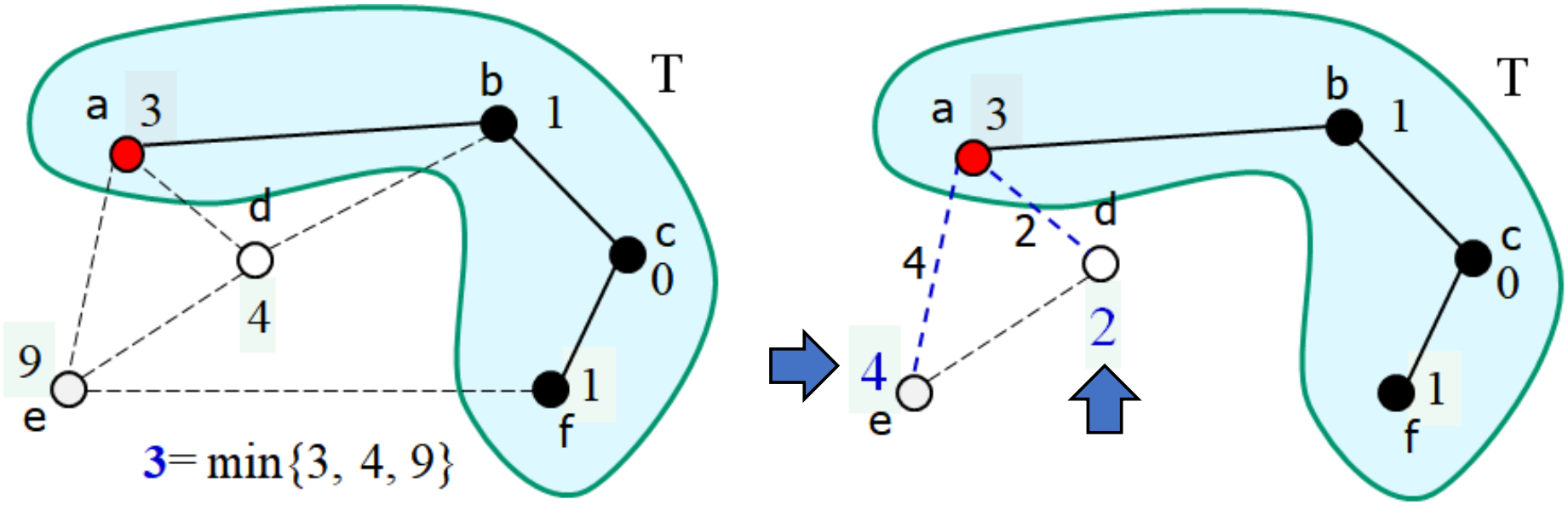
정점 d와 f를 잇는 간선의 가중치 (7)가 b와 d를 잇는 가중치(4)보다 크므로 갱신되지 않음.



Vertex	a	b	c	d	e	f
Distance	3	1	0	4	**9**	1

Prim 알고리즘의 수행 과정 예시 (8/11)

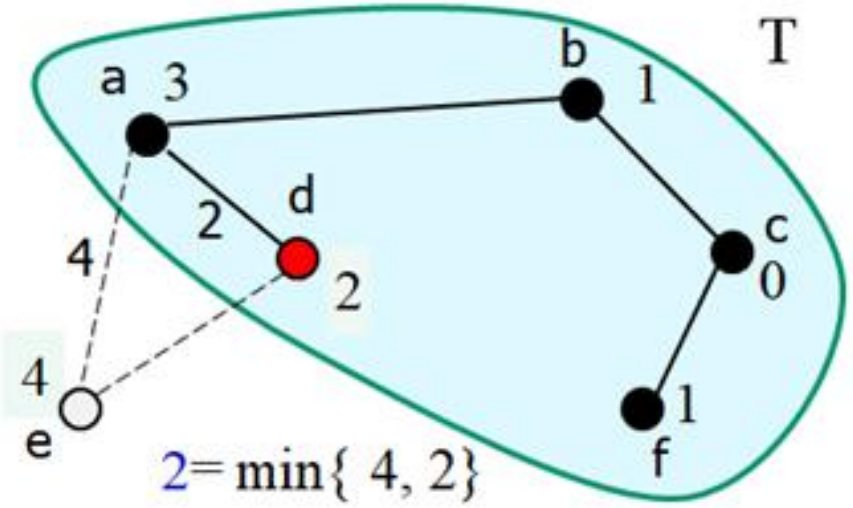
- 정점 a를 T에 추가한 후 정점 a와 연결된 정점 e와 d에 대한 거리 D[e], D[d]를 갱신한다.

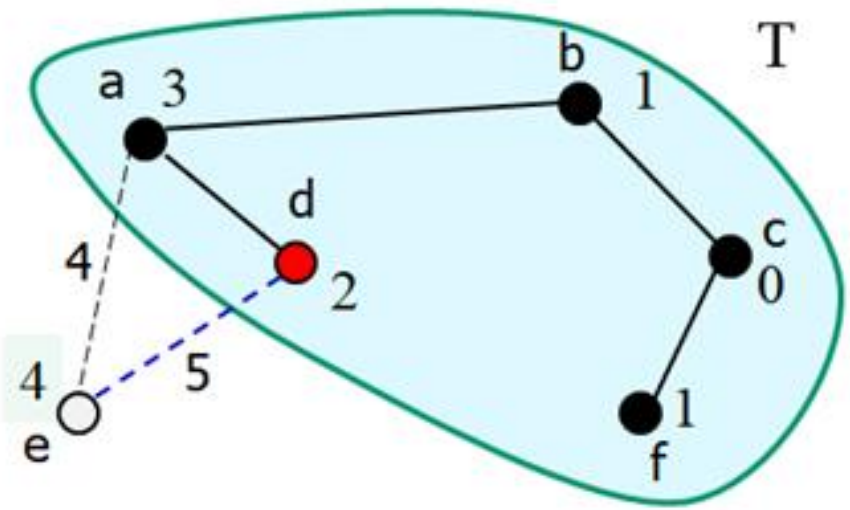


Vertex	a	b	c	d	e	f
Distance	3	1	0	**2**	**4**	1

Prim 알고리즘의 수행 과정 예시 (9/11)

- 최소 가중치 간선을 갖는 정점 d를 T에 추가한다.

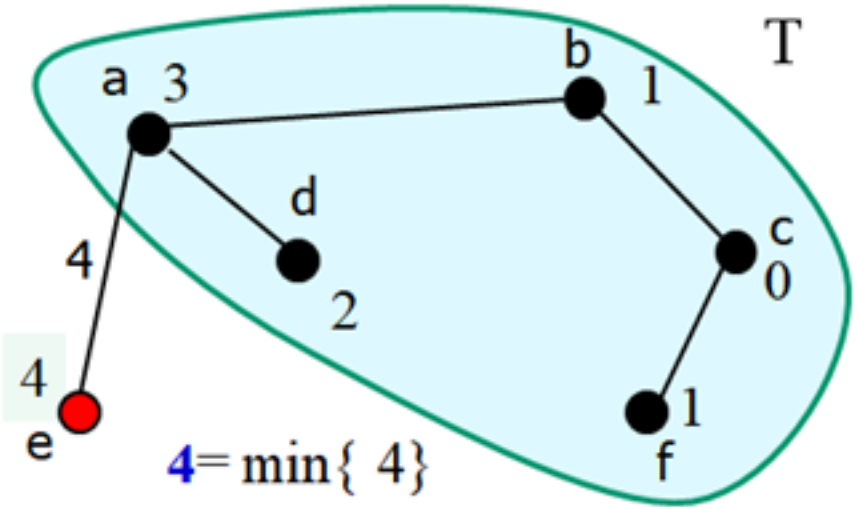


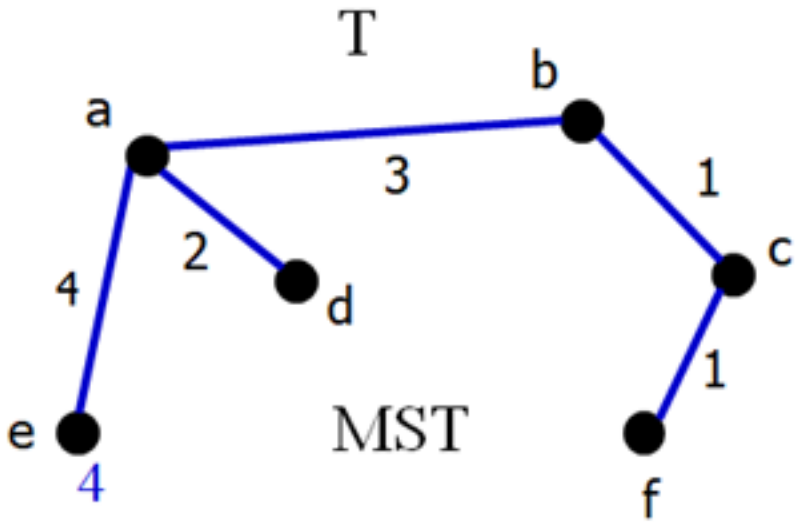


Vertex	a	b	c	d	e	f
Distance	3	1	0	2	4	1

Prim 알고리즘의 수행 과정 예시 (10/11)

- 정점 e를 T에 추가한다.
- **Line 13:** 모든 정점이 T에 추가되었으므로 T는 MST이다.

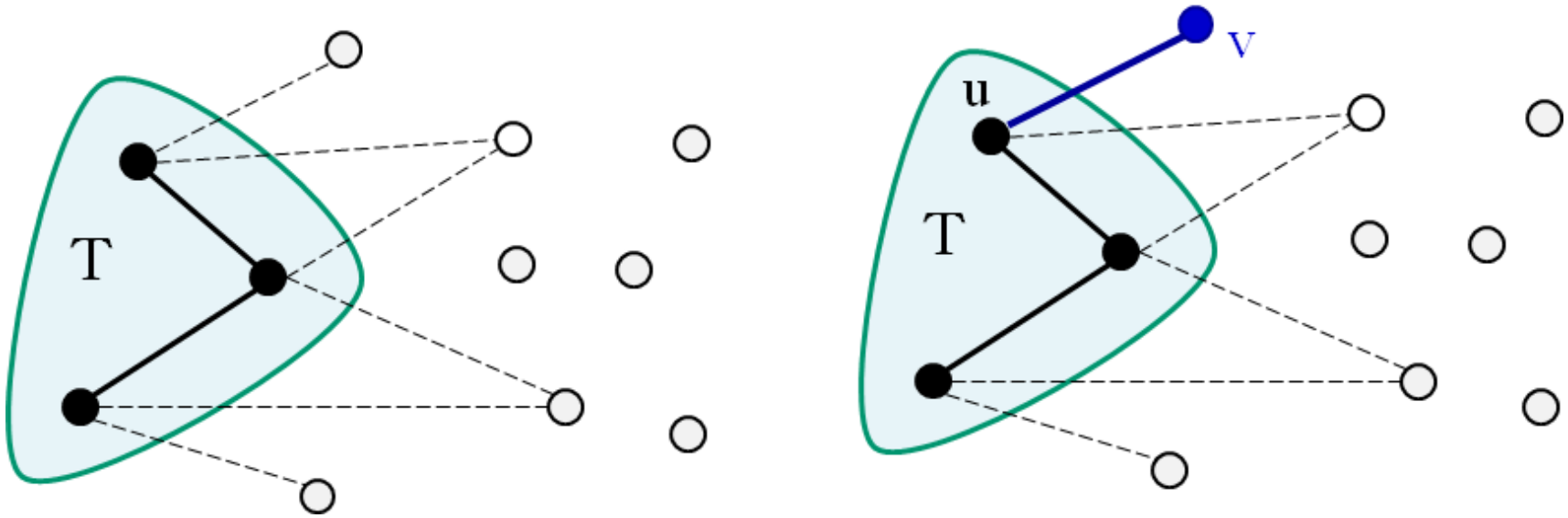




Vertex	a	b	c	d	e	f
Distance	3	1	0	2	4	1

Prim 알고리즘의 수행 과정 예시 (11/11)

- PrimMST가 찾은 T에는 왜 사이클이 없을까?
  - ➢ Prim 알고리즘은 **집합 T 밖에 위치한 정점**만을 추가하므로 사이클이 만들어지지 않는다.



Prim 알고리즘의 시간 복잡도

- While 반복문은 $n-1$회 수행된다.
  - 1회 반복될 때 line 9에서 T에 속하지 않은 각 점 v에 대하여, D[v]가 최소인 점 v_{min}을 찾는데 최대 $\boldsymbol{O(n)}$의 시간이 소요된다.
  - 이는 배열 D에서 (현재 T에 속하지 않은 점들에 대해서) 최소값을 찾는 것이고, 배열의 크기는 n이기 때문이다.
- 이에 따라, Prim 알고리즘의 시간 복잡도를 정리하면…
  - 위의 while 반복문에서 소요되는 시간은 $(n-1) \times O(n) = \boldsymbol{O(n^2)}$
  - 최소 힙(Binary Heap)을 사용하여 v_{min}을 찾으면 $\boldsymbol{O(m \log n)}$이다. (※ m = 간선의 수)
  - 따라서 간선의 수가 $O(n)$ 이면, 총 시간 복잡도는 $\boldsymbol{O(n \log n)}$이다.

Kruskal과 Prim 알고리즘의 수행 과정 비교

- Kruskal 알고리즘
  - ➢ 간선이 1개씩 T에 추가되는데, 이는 마치 n개의 점들이 각각의 트리인 상태에서 간선이 추가되면 2개의 트리가 1개의 트리로 합쳐지는 것과 같다.
  - ➢ Kruskal 알고리즘은 이를 반복하여 1개의 트리인 T를 생성한다.
  - ➢ **n개의 트리들이 점차 합쳐지면서** 1개의 신장 트리가 완성된다.

- Prim 알고리즘
  - ➢ T가 점 1개인 트리에서 시작되어 간선을 1개씩 추가한다.
  - ➢ **1개의 트리가 점차적으로 자라나면서** 신장 트리가 완성된다.

최소 비용 신장 트리의 응용

- 최소 비용으로 선로 또는 파이프 네트워크(ex. 인터넷 광 케이블 선로, 케이블 TV선로, 전화선로, 송유관로, 가스관로, 배수로 등)를 설치하기 위한 경로를 계획하는 데 활용된다.

▶ 최소 신장 트리 알고리즘은 최소 비용으로 선로(왼쪽)와 송유관로(오른쪽)를 설치하는 데 활용된다.

Q & A